

NLP - Homework 4

Sravanth Chowdary Potluri
und5uv

December 3, 2024

Question 1

Based on the description in (Vaswani et al., 2017), please describe the major difference between the Transformer encoder and the Transformer decoder. As we explained in class, both encoder and decoder consists of the major components, such as feed-forward module and multi-head attention. Your answer should cover the following aspects

- What is the major module difference between the Transformer encoder and the Transformer decoder?

Answer :- The major difference between the Transformer encoder as per my interpretation is that the decoder contains a "*Masked Multi-Head Attention*" layer. Other differences include The Decoder having a cross-attention module, which allows the decoder to attend to the encoder's output representations, enabling integration of contextual information from the source sequence.

- How that module is implemented?

Answer:- The Masked Multi-Head Attention layer in the decoder is implemented similar to the one in the encoder for the most part. However The Self-Attention layers in the Masked Multi-Head Attention are different, They only allow each position in the decoder to attend to all positions in the decoder up to and including that position, And not the tokens that come after that position. We prevent this leftward information flow from the tokens that are on the right to preserve the auto-regressive property which means that the prediction of each subsequent token is only influenced by the tokens that are previously in the sequence. This feature is implemented by using an upper triangular mask that sets all values in the input of the softmax which correspond to illegal connections to $-\infty$, here illegal connections are the connections between the future tokens and the tokens before

it. 1 displays the application of the mask to the Softmax's Input. This ensures that after applying the softmax function, these masked scores effectively become 0, preventing attention to those positions.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + \text{Mask} \right) V \quad (1)$$

For a sequence of length $n = 6$, the mask matrix for position t would look like this:

$$\text{Mask} = \begin{bmatrix} 0 & -\infty & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty & -\infty \\ 0 & 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (2)$$

Meaning that, at each position, the model can only attend to itself and preceding positions.

- **Why the difference is critical, e.g., for machine translation?**

Answer:- This difference is critical In Machine Translation where we translate a sentence word by word from a source language to a target language in a sequential manner. Since Machine translation involves generating the target sentence token by token. If the decoder could see future tokens, it would “cheat,” learning to predict words based on context it wouldn't have during inference. During training, the model is exposed to the full target sequence, but during inference, it has to generate tokens sequentially without having access to future tokens. The masking that we do during training mimics the conditions of inference, ensuring the model generalizes well and follows the Autoregressive Property. Without masking, the decoder could focus on future tokens during training, leading to overfitting and poor performance during inference.

Question 2

Comparison between the BERT model and the GPT model (GPT-1, to be specific). Although both of them are based on the Transformer architecture. Their difference can be categorized at least in the following three aspects.

- **Model architecture:** following your answer to the previous question, please identify which specific Transformer (e.g., encoder, decoder, or both) is used in each of the two models.

Answer:-

BERT: Uses only the Transformer encoder to capture bidirectional context.

GPT-1: Uses only the Transformer decoder to facilitate autoregressive language modeling, processing text in a unidirectional (left-to-right) manner.

- **Pre-training strategy:** please summarize the training strategies used in each of these two models. Note that, your answer should not include fine-tuning strategies.

Answer:- The Pretraining Strategies used In BERT and GPT-1 are.

BERT:

Masked Language Modeling (MLM): Randomly masks tokens in the input and trains the model to predict these tokens based on context from both directions.

Next Sentence Prediction (NSP): Trains the model to predict if a given sentence is the subsequent sentence in the original text, helping it understand sentence relationships.

GPT-1:

Pre-trained using **Causal Language Modeling (CLM)**, Which Trains the model to predict the next word in a sequence, using all previous words as context. This unidirectional approach models the probability distribution of the next word given the sequence. It is called Causal Language Modelling because each prediction for the next word is Caused by the words preceeding it, ensuring the causal relationship and preserving the autoregressive property.

- **Applications:** what are the difference of these two models regarding NLP tasks. Please list the NLP applications that can highlight the difference between these two models. As a counter-example, “text classification” is not a good answer, as both of them can be used for classification. There are many NLP tasks, and your answer can only be the tasks that we described (at least, briefly) in class, including sentiment classification, topic classification, word embedding, language modeling, question answering, natural language inference, semantic role labeling, machine translation, text generation, conversation modeling.

Answer:- Some of the tasks in the given list of tasks in the question that I believe can distinguish between a decoder-only model like GPT-1 and an encoder-only model like BERT, and highlight the differences, are:

- **Word Embedding (BERT):** BERT is better suited for word embedding applications because it captures the context of words in a *bi-directional manner*. This allows BERT to generate embeddings that are sensitive to both the left and right contexts of a word, leading to more nuanced and semantically meaningful embeddings. GPT-1, being uni-directional, lacks this capability and is less effective for generating high-quality word embeddings.

- **Language Modeling (GPT-1):** GPT-1 excels at language modeling because it is designed as an *auto-regressive decoder*, predicting the next word in a sequence based on previous tokens. This makes it particularly suitable for tasks that involve generating coherent and contextually appropriate text, such as text completion and continuation.
- **Natural Language Inference (BERT):** BERT is better for natural language inference (NLI) tasks because its *encoder-based architecture* allows for a thorough understanding of relationships between sentences. Its bi-directional nature enables it to evaluate semantic entailment, contradiction, or neutrality by analyzing sentence pairs holistically.
- **Semantic Role Labeling (BERT):** BERT is well-suited for semantic role labeling as it can capture detailed relationships between different parts of a sentence. Its ability to understand the “who did what to whom” structure in a sentence benefits greatly from its bi-directional context comprehension.
- **Text Generation (GPT-1):** GPT-1 is inherently better for text generation because of its *auto-regressive nature*. It generates sequences token by token, considering previous tokens to maintain coherence and logical flow, which is crucial for creative tasks like storytelling, dialogue continuation, or general text generation.
- **Conversational Modeling (GPT-1):** GPT-1 is more effective in conversational modeling as it is trained to predict the next token in a sequence, making it highly suitable for generating contextually relevant and human-like responses in dialogue systems. BERT, being an encoder, lacks the generative capability necessary for conversational tasks.

BERT is primarily designed for *natural language understanding (NLU)* tasks requiring deep contextual comprehension, while GPT-1 is tailored towards *natural language generation (NLG)* tasks that involve producing text. This distinction stems from their architectures—BERT’s encoder-only model processes input sequences holistically, whereas GPT-1’s decoder-only model processes sequences sequentially, generating text in an auto-regressive manner.

Question 3

Prefix Tuning. In the topic of efficient fine-tuning, we talked about prefix tuning as one of the fine-tuning techniques. The basic idea of prefix-tuning is to introduce additional latent representations to each layer. During fine-tuning, these representations will be updated for task-specific applications.

- **Number of parameters introduced by prefix-tuning.** Assume we are using prefix-tuning to fine-tune the GPT model. Given the dimension of latent

representations is 768 and the number of prefix representations for each layer is 4, what are the total number of parameters introduced by prefix tuning? (Hint: only consider the case without using feed-forward NN for projection.)

Answer:-

For each layer, prefix-tuning introduces 4 prefix representations of dimension 768.

Parameters per layer: $4 \times 768 = 3,072$.

We know from the original paper that the GPT-1 Model has 12 layers of decoder-only transformers.

Total parameters introduced (without projection and 12 layers) by prefix tuning:

$$\text{Total Parameters} = 3,072 \times 12 = 36864 \text{ Parameters}$$

- **Although the lack of theoretical justification, we can still explain the performance of prefix tuning by comparing it with discrete prompt optimization. What are the two main reasons that prefix tuning may outperform discrete prompt optimization in practice?**

Answer:-

The two main reasons can be explained from two perspectives

1.From the Initialization Perspective:

Prefix tuning initializes embeddings (latent representations) that integrate naturally into the pre-trained model's architecture. As discussed in the class, we can initialize the prefix embeddings with words relevant to the task, which produce better model performance as well. These embeddings provide a meaningful starting point as they align well with the model's learned numerical vector space, unlike discrete prompts, which can start with natural language tokens that might not be always relevant to the task we are trying to accomplish.

2.From the Optimization Perspective:

Prefix tuning involves optimizing numerical vectors, which allows for smoother updates in the continuous parameter space. This is more effective than the trial-and-error approach of finding optimal discrete tokens in prompt engineering. Discrete prompt optimization relies on selecting from a fixed set of natural language tokens and involves no training examples. It often requires extensive manual tuning or iterative exploration, which is less efficient compared to fine-tuning embeddings with training data in prefix tuning. Even though, prefix tuning requires samples for training.

Question 4

Multi-head Attention. Consider the procedure of computing multi-head attention (lecture 11, page 8).

- Assume the query vector, key vector and value vector are all 512 dimensions. If we use multiple heads with $H = 8$, what are the dimensions of the three metrics $\{W_i^Q, W_i^K, W_i^V\}$ for each head i ?

Answer:-

We are given:

- Query, key, and value vectors have dimensions of 512.
- Number of heads $H = 8$.

For multi-head attention, each head processes a fraction of the total dimensionality. Thus, the dimensionality per head is:

$$d_{\text{head}} = \frac{512}{8} = 64$$

The weight matrices W_i^Q, W_i^K, W_i^V are used to project the query, key, and value vectors into subspaces for each head. Therefore, for each head i , the dimensions of these matrices are:

$$W_i^Q \in \mathbb{R}^{512 \times 64}, \quad W_i^K \in \mathbb{R}^{512 \times 64}, \quad W_i^V \in \mathbb{R}^{512 \times 64}$$

Therefore the dimensions of all the weight matrices per head are 512×64

- Now, consider the collection $\{W_i^Q\}$, if we define W^Q as the stack of $\{W_i^Q\}$, then what is the dimensions of W^Q ?

Answer:-

The matrix W^Q is formed by stacking the individual W_i^Q matrices for all $H = 8$ heads. Since each W_i^Q has dimensions 512×64 , the resulting matrix W^Q has dimensions:

$$W^Q \in \mathbb{R}^{512 \times (8 \cdot 64)}$$

Simplifying:

$$W^Q \in \mathbb{R}^{512 \times 512}$$

The Stacking produces a matrix of size 512×512 , just like the the concatenation process after computing Multi Head Self Attention.

Question 5

LoRA. The original proposal of using LoRA is only on the attention related metrics. Let us consider W^Q only in the following questions.

- Based on the definition of LoRA, what are the dimensions of the adapter ΔW^Q for W^Q ?

Answer:-

ΔW^Q has the same dimensions as W^Q . If W^Q has dimensions $d \times k$ then ΔW^Q also has the dimensions $d \times k$

- If we assume the dimensions of the query vector are 512, for low-rank estimation with $r = 4$, what are the dimensions of A and B in $\Delta W^Q = A \cdot B$?

Answer:-

If the query vector has 512 dimensions, then W^Q has a shape of 512×512 .

LoRA decomposes $\Delta W^Q = A \cdot B$, where:

- A dimensions = 512×4 .
- B dimensions = 4×512 .

At rank $r = 4$

References

1. Vaswani, A. "Attention is all you need." Advances in Neural Information Processing Systems (2017).
2. Devlin, Jacob. "Bert: Pre-training of deep bidirectional transformers for language understanding." arXiv preprint arXiv:1810.04805 (2018).
3. Radford, Alec. "Improving language understanding by generative pre-training." (2018).
4. Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." arXiv preprint arXiv:2106.09685 (2021).
5. Li, Xiang Lisa, and Percy Liang. "Prefix-tuning: Optimizing continuous prompts for generation." arXiv preprint arXiv:2101.00190 (2021).
6. Made use of GPT-4o for Better clarity of answering.